Worksheet: trace it before you run it

NAME _____ DATE 16 September 2026

SECTION 1 · 10 MINUTES

Opener

One line each. What does the line do? If you are not sure, write that.

```
for i in range(10):
```

```
total = total + r
```

```
if r > best:
```

```
slot = (slot + 5) % 9
```

SECTION 2 · 25 MINUTES

The cold read

The first half

Read the program. Then fill the table one row per pass through the loop. The top row shows the values before the loop starts. In the two tinted columns write **yes** or **no**: did the gate open on this pass? Three rows are done for you, shaded, so you can check your working against them as you go. The rows in between are yours.

SWEEP_REPORT.PY, FIRST HALF

```
readings = [41, 58, 33, 58, 12, 77, 58, 60, 29, 77]
limit = 55

t = 0
c = 0
b = 0
bi = 0

for i in range(10):
    r = readings[i]

    t = t + r

    if r > limit:
        c = c + 1

    if r > b:
        b = r
        bi = i

avg = t / 10

print(f"total {t}")
print(f"average {avg}")
print(f"over {limit}: {c}")
print(f"best {b} at index {bi}")
```

i	r	t	r > limit?	c	r > b?	b	bi
start		0		0		0	0
0	41	41	no	0	yes	41	0
1	58						
2	33						
3	58	190	yes	2	no	58	1
4	12						
5	77						
6	58						
7	60	397	yes	5	no	77	5
8	29						
9	77						

Now write the four lines the program will print.

`total`

`average`

`over 55:`

`best ... at index ...`

The list holds 77 twice, at index 5 and at index 9, and the fourth line names only one of them. The gate `if r > b:` decides which. Circle the one character in that gate that makes the comparison, then write one sentence: when the second 77 arrives, will the gate say yes or no, and why?

The second half

`%` gives the remainder after dividing. `41 % 9` is 5, because 41 is four nines with 5 left over. Work the middle column first, then take the remainder. Rows 3, 6, and 9 are done for you, shaded. The first three rows are required, and row 3 is there to check yourself against. Finish the rest if you have time.

SWEEP_REPORT.PY, SECOND HALF

```
slot = 0
for i in range(10):
    slot = (slot + readings[i]) % 9
    print(f"i={i} reading={readings[i]} slot={slot}")
```

i	reading	slot + reading	% 9 = slot
start			0
0	41		
1	58		
2	33		
3	58	64	1
4	12		
5	77		
6	58	58	4
7	60		
8	29		
9	77	80	8

Rows 1 and 3 both have a reading of 58. Do they land on the same slot? Say why or why not in one sentence.

SECTION 3 · 10 MINUTES

Run it

Now run the program. Copy the real output next to your prediction. Mark each line right or wrong. Do not erase a wrong prediction.

Line	What I predicted	What it printed	Right?
`total`			
`average`			
`over 55`			
`best`			
`slot, i=0 to 2`			

SECTION 4 · 30 MINUTES

The style pass

The program works and it is hard to read. Rename every short name so the name says what the value is. Yours does not have to match anyone else's. It has to be a name the next reader understands without scrolling back up.

Old name	What the value is, in words	Clearer new name
`t`		
`c`		
`b`		
`bi`		
`r`		
`avg`		
`i`		

Then the docstring. A docstring is one sentence, inside triple quotes, on line 1 of the file, saying what the program is for. Python ignores it. The next reader does not. Here is one for this program:

LINE 1 OF SWEEP_REPORT.PY

```
"""Summarize a run of tower readings and report where the strongest one sat."""
```

Copy it onto line 1 of your file. If you would rather say it your own way, write your sentence here first, then type it between the quotes:

A second one, harder

This program walks along a corridor of doors, where 1 means open and 0 means shut. Trace all four variables, then name them. A hint: every one of the four variables is counting something about the doors, so every new name should have the word door or open or shut in it. Watch the `z` column as you fill it in. It climbs and drops back to 0, and the name has to say why. Rows 1, 5, and 9 are done for you, shaded, to check against. One of the four variables is much harder to name than the others.

GATE_LOG.PY

```
door = [1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0]

x = 0
y = 0
z = 0
q = 0

for i in range(12):
    d = door[i]

    if d == 1:
        x = x + 1
        z = z + 1

    if d == 0:
        y = y + 1
        z = 0

    if z > q:
        q = z

print(f"x {x}")
print(f"y {y}")
print(f"z {z}")
print(f"q {q}")
```

i	d	x	y	z	q
start		0	0	0	0
0	1				
1	0	1	1	0	1
2	1				
3	1				
4	0				
5	0	3	3	0	2
6	1				
7	0				
8	1				
9	1	6	4	2	2
10	1				
11	0				

Old name	What the value is, in words	Clearer new name
x		
y		
z		
q		

Which of the four variables was hardest to name, and why?